Senior AI Engineering Curriculum

Week 2 · Day 12: Caching, Cost Optimization & API Layer

Prefix Caching, Semantic Caching, Prompt Compression, API Gateway & Multi-Model Routing

---

Goal: Cut inference cost by 40-60% through a layered stack of caching and routing strategies: exact prefix caching at the KV level, semantic similarity caching at the embedding level, prompt compression to reduce token count, an API gateway for auth/rate-limiting/logging, fallback routing for resilience, and multi-model routing to match query complexity to model cost.

Context: At production scale, LLM inference is expensive. A single A100 costs $2-4/hr on cloud; a 70B model serving 100 QPS burns thousands of dollars per day. This day is about the engineering discipline of spending only what each request actually needs -- and never spending twice on the same computation.

PART A — Prefix & Prompt Caching

Two Levels of Prefix Caching

Prefix caching exists at two independent levels in the stack. Understanding the distinction is critical because they have different cache scopes, hit rates, and implementation complexity:

Level	Where	What is cached	Hit condition	Scope
KV-level (vLLM)	GPU VRAM	Key/Value tensors for tokens	Exact token-ID match	Within one server process
Response-level	Redis / disk	Full generated text	Exact prompt string match	Across server restarts
Semantic	Vector DB/RAM	Embedding + cached response	Cosine similarity >= T	Across users, sessions

A.1 vLLM KV-Level Prefix Caching (APC) Deep-Dive

The vLLM Automatic Prefix Cache (APC) hashes each fully-filled KV block (block_size=16 tokens by default) using the concatenated token IDs of that block plus all ancestor blocks. This hash becomes the block's cache key. On a new request, vLLM computes hashes for all prompt blocks and performs a lookup in the block hash table. Matched blocks are reused directly -- their KV data is already in GPU VRAM and the prefill computation is completely skipped for those tokens.

```
# Enable and tune APC in vLLM
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
    --enable-prefix-caching \
    --enable-chunked-prefill \         # process prompts in chunks (required for APC)
    --max-num-batched-tokens 2048 \    # chunk size: tokens processed per prefill step
    --dtype float16 \
    --gpu-memory-utilization 0.92 \    # extra VRAM -> more blocks -> better cache retention
    --port 8000
```

```
# Python API with APC
from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    enable_prefix_caching=True,
    enable_chunked_prefill=True,
    max_num_batched_tokens=2048,
    dtype="float16",
    gpu_memory_utilization=0.92,
)

# Craft system prompt designed for maximum cache reuse
# Rule: the cacheable prefix must end on a block boundary (multiple of block_size=16)
# Pad with spaces if needed to align to the next 16-token boundary

from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3-8B-Instruct")
SYSTEM = (
    "You are a senior AI infrastructure engineer with expertise in vLLM, "
    "Kubernetes, distributed training, and LLM quantization. "
    "Always respond concisely with code examples where relevant. "
)

# Check token count and alignment
n_sys_tokens = len(tok.encode(SYSTEM))
block_size   = 16
remainder    = n_sys_tokens % block_size
print(f"System prompt tokens: {n_sys_tokens}")
print(f"Remainder mod {block_size}: {remainder}")
if remainder:
    print(f"Pad by {block_size - remainder} tokens for optimal cache alignment")
```

A.2 Response-Level Exact Cache with Redis

```
# Exact response cache: hash the full prompt string, store/retrieve response
# Extremely fast (sub-millisecond), zero compute cost on hit
# Use for: FAQs, fixed template queries, idempotent tool calls

import redis, hashlib, json, time
from openai import OpenAI

r      = redis.Redis(host="localhost", port=6379, db=0, decode_responses=True)
client = OpenAI(base_url="http://localhost:8000/v1", api_key="vllm")

CACHE_TTL = 3600  # seconds: 1 hour

def cache_key(messages: list, model: str, max_tokens: int) -> str:
    payload = json.dumps({"messages": messages, "model": model,
                          "max_tokens": max_tokens}, sort_keys=True)
    return "llm_cache:" + hashlib.sha256(payload.encode()).hexdigest()

def cached_completion(messages: list, model="llama3",
                      max_tokens=200, temperature=0.0) -> dict:
    # temperature=0 required for caching -- stochastic outputs should not be cached
```

```
    if temperature > 0:
        raise ValueError("Only cache deterministic (temperature=0) responses")

    key    = cache_key(messages, model, max_tokens)
    cached = r.get(key)

    if cached:
        data = json.loads(cached)
        data["cache_hit"] = True
        return data

    # Cache miss -- call the model
    t0   = time.perf_counter()
    resp = client.chat.completions.create(
        model=model, messages=messages,
        max_tokens=max_tokens, temperature=0.0,
    )
    elapsed = time.perf_counter() - t0

    result = {
        "content"          : resp.choices[0].message.content,
        "prompt_tokens"    : resp.usage.prompt_tokens,
        "completion_tokens": resp.usage.completion_tokens,
        "model"            : model,
        "elapsed_s"        : round(elapsed, 3),
        "cache_hit"        : False,
    }
    r.setex(key, CACHE_TTL, json.dumps(result))
    return result

# Test cache behaviour
MSGS = [{"role": "user", "content": "What is PagedAttention? Answer in 2 sentences."}]

r1 = cached_completion(MSGS)
r2 = cached_completion(MSGS)

print(f"Call 1 -- hit={r1['cache_hit']}  elapsed={r1['elapsed_s']}s")
print(f"Call 2 -- hit={r2['cache_hit']}  elapsed={r2.get('elapsed_s','<1ms')}")
print(f"Response: {r1['content'][:120]}")
```

PART B — Semantic Caching with FAISS

Why Exact Caching Is Not Enough

Exact caching only hits when prompts are byte-for-byte identical. In practice, users rephrase the same question constantly: 'What is KV caching?', 'Explain KV cache', 'How does the KV cache work?' These are semantically identical but produce three cache misses with exact matching. Semantic caching embeds each prompt into a vector space and uses approximate nearest neighbour (ANN) search to find similar past prompts. If the closest match exceeds a similarity threshold, the cached response is returned directly.

Semantic Cache Architecture

Component	Role	Tool
Embedding model	Convert prompt text to dense vector	sentence-transformers (local) or OpenAI ada-002
Vector index	Store + retrieve similar prompt vectors	FAISS (in-process) or Qdrant/Weaviate (server)
Similarity metric	Measure semantic closeness	Cosine similarity (inner product on L2-normed vectors)
Response store	Store full response for each cached prompt	Redis or in-memory dict
Threshold	Minimum similarity for a cache hit	0.90-0.95 for factual tasks, 0.85 for chat

B.1 FAISS Semantic Cache Implementation

```
# pip install faiss-cpu sentence-transformers numpy
import faiss, numpy as np, json, time, hashlib
from sentence_transformers import SentenceTransformer
from openai import OpenAI
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class CacheEntry:
    prompt:   str
    response: str
    tokens:   int
    hit_count: int = 0

class SemanticCache:
    def __init__(
        self,
        embed_model: str = "all-MiniLM-L6-v2",   # 384-dim, fast, good quality
        similarity_threshold: float = 0.92,
        max_entries: int = 10_000,
    ):
        self.embedder   = SentenceTransformer(embed_model)
        self.threshold  = similarity_threshold
        self.max_entries = max_entries
        self.dim        = self.embedder.get_sentence_embedding_dimension()

        # FAISS IndexFlatIP: exact inner product search on L2-normed vectors
        # = cosine similarity search (fast for <100k entries)
```

```
        self.index      = faiss.IndexFlatIP(self.dim)
        self.entries    : list[CacheEntry] = []
        self.stats      = {"hits": 0, "misses": 0, "tokens_saved": 0}

    def _embed(self, text: str) -> np.ndarray:
        vec = self.embedder.encode([text], normalize_embeddings=True)
        return vec.astype(np.float32)

    def get(self, prompt: str) -> Optional[CacheEntry]:
        if self.index.ntotal == 0:
            self.stats["misses"] += 1
            return None

        query_vec = self._embed(prompt)
        scores, indices = self.index.search(query_vec, k=1)
        score = float(scores[0][0])
        idx   = int(indices[0][0])

        if score >= self.threshold and 0 <= idx < len(self.entries):
            entry = self.entries[idx]
            entry.hit_count += 1
            self.stats["hits"] += 1
            self.stats["tokens_saved"] += entry.tokens
            return entry

        self.stats["misses"] += 1
        return None

    def put(self, prompt: str, response: str, tokens: int):
        if len(self.entries) >= self.max_entries:
            return   # simple eviction: stop accepting new entries at limit
        vec = self._embed(prompt)
        self.index.add(vec)
        self.entries.append(CacheEntry(prompt=prompt, response=response, tokens=tokens))

    def hit_rate(self) -> float:
        total = self.stats["hits"] + self.stats["misses"]
        return self.stats["hits"] / total if total > 0 else 0.0

    def cost_savings(self, cost_per_1k_tokens: float = 0.002) -> float:
        return self.stats["tokens_saved"] / 1000 * cost_per_1k_tokens

# ■■ Usage with vLLM ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
cache  = SemanticCache(similarity_threshold=0.92)
client = OpenAI(base_url="http://localhost:8000/v1", api_key="vllm")

def smart_completion(prompt: str, model="llama3", max_tokens=150) -> dict:
    t0    = time.perf_counter()
    entry = cache.get(prompt)

    if entry:
        return {
            "response"  : entry.response,
            "cache_hit" : True,
            "elapsed_ms": round((time.perf_counter() - t0) * 1000, 2),
            "tokens"    : 0,   # no tokens consumed
```

```
        }

    resp = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        max_tokens=max_tokens, temperature=0.0,
    )
    content = resp.choices[0].message.content
    tokens  = resp.usage.total_tokens
    cache.put(prompt, content, tokens)

    return {
        "response"  : content,
        "cache_hit" : False,
        "elapsed_ms": round((time.perf_counter() - t0) * 1000, 2),
        "tokens"    : tokens,
    }

# Test semantic similarity hits
queries = [
    "What is KV caching?",                     # original
    "Explain the KV cache mechanism.",          # paraphrase -> should hit
    "How does key-value caching work in LLMs?", # paraphrase -> should hit
    "What is PagedAttention?",                  # different topic -> miss
    "Describe KV cache in transformers.",       # paraphrase -> should hit
]
for q in queries:
    r = smart_completion(q)
    print(f"{'HIT ' if r['cache_hit'] else 'MISS'} [{r['elapsed_ms']:>6.1f}ms] "
          f"[{r['tokens']:>5} toks] {q[:55]}")

print(f"
Hit rate     : {cache.hit_rate():.1%}")
print(f"Tokens saved : {cache.stats['tokens_saved']:,}")
print(f"Cost saved   : ${cache.cost_savings():.4f}")
```

B.2 Threshold Tuning and Quality Evaluation

```
from sentence_transformers import SentenceTransformer
import numpy as np

embedder = SentenceTransformer("all-MiniLM-L6-v2")

def cosine_sim(a: str, b: str) -> float:
    vecs = embedder.encode([a, b], normalize_embeddings=True)
    return float(np.dot(vecs[0], vecs[1]))

# Calibration pairs: (query_a, query_b, should_hit)
calibration = [
    # True positives -- semantically identical, should hit
    ("What is KV caching?",            "Explain KV cache",                    True),
    ("How does PagedAttention work?",  "Describe PagedAttention mechanism",   True),
    ("List vLLM startup flags",        "What are vLLM configuration options?", True),

    # True negatives -- different meaning, should NOT hit
    ("What is KV caching?",            "What is quantization?",               False),
```

```
    ("Explain tensor parallelism",     "What is pipeline parallelism?",       False),
    ("How do I install vLLM?",         "How do I uninstall vLLM?",            False),

    # Edge cases -- borderline similar
    ("What is flash attention?",       "What is attention in transformers?",   None),
    ("How to reduce VRAM usage?",      "How to optimize GPU memory?",          None),
]

print(f"{'Query A':<40} {'Query B':<40} {'Sim':>6} {'Should hit'}")
print("-" * 95)
for a, b, expected in calibration:
    sim = cosine_sim(a, b)
    hit_090 = sim >= 0.90
    hit_092 = sim >= 0.92
    hit_095 = sim >= 0.95
    exp_str = "YES" if expected else ("NO" if expected is False else "?")
    print(f"{a[:38]:<40} {b[:38]:<40} {sim:>6.3f}  {exp_str}")

# Recommendation: plot precision-recall curve over calibration set,
# pick threshold at F1 maximum for your specific workload
print("
Guideline thresholds:")
print("  0.95+  ->  Very conservative (almost exact match only)")
print("  0.92   ->  Recommended default for factual/technical queries")
print("  0.88   ->  Aggressive (good for high-repetition chatbots)")
print("  0.85   ->  Too loose for most use cases (quality risk)")
```

PART C — Prompt Compression with LLMLingua

Why Compress Prompts?

For RAG pipelines, agent tool schemas, and long-context summarization tasks, prompts routinely contain thousands of tokens. At $0.002/1K tokens (typical self-hosted GPU cost), a 4,000-token prompt at 100 QPS costs $28,800/day -- just for the prompt tokens, before any generation. Prompt compression removes redundant content from long contexts while preserving the information needed for the model to answer correctly. The state-of-the-art tool is LLMLingua (Microsoft Research, 2023).

How LLMLingua Works

LLMLingua uses a small language model (e.g. LLaMA-2 7B or GPT-2) as a **compressor proxy** to score each token's perplexity given its context. High-perplexity tokens (surprising, information-dense) are kept. Low-perplexity tokens (predictable, redundant) are dropped. The compression is applied at three granularities: instruction-level (coarse), document-level (medium), and token-level (fine). LLMLingua-2 (2024) uses a BERT-style classifier trained to predict token retention, achieving better quality at higher compression ratios.

C.1 LLMLingua Installation and Basic Compression

```
# pip install llmlingua transformers
from llmlingua import PromptCompressor
import time

# Initialise compressor (downloads a small proxy LM ~3GB on first run)
compressor = PromptCompressor(
    model_name="microsoft/llmlingua-2-bert-base-multilingual-cased-meetingbank",
    use_llmlingua2=True,    # LLMLingua-2 (better quality than v1)
    device_map="cuda",
)

# Typical RAG prompt structure
INSTRUCTION = "Answer the question based only on the provided context."

CONTEXT = (
    "PagedAttention manages KV cache using fixed-size blocks and a block table "
    "that maps logical to physical GPU memory blocks. This eliminates fragmentation, "
    "enables copy-on-write sharing for beam search, and allows larger batch sizes. "
    "vLLM showed 2-4x higher throughput than HuggingFace Transformers by combining "
    "PagedAttention with continuous batching."
) * 20   # simulate a longer RAG document

QUESTION = "How does PagedAttention reduce memory fragmentation?"

# Full (uncompressed) prompt
full_prompt = f"{INSTRUCTION}

Context:
{CONTEXT}

Question: {QUESTION}
```

```
Answer:"

# Compress at different ratios
for ratio in [0.5, 0.33, 0.25]:
    t0 = time.perf_counter()
    result = compressor.compress_prompt(
        context=[CONTEXT],
        instruction=INSTRUCTION,
        question=QUESTION,
        rate=ratio,                    # target compression ratio (keep this fraction)
        target_token=None,             # or set explicit target token count
        condition_compare=True,        # compare with uncompressed for quality
        dynamic_context_compression_ratio=0.3,
    )
    elapsed = time.perf_counter() - t0

    orig_tokens = result["origin_tokens"]
    comp_tokens = result["compressed_tokens"]
    actual_ratio = comp_tokens / orig_tokens

    print(f"Target ratio={ratio:.2f}: "
          f"{orig_tokens} -> {comp_tokens} tokens  "
          f"(actual {actual_ratio:.2f})  "
          f"[{elapsed*1000:.0f}ms compression]")
    print(f"  Compressed context (first 200 chars): "
          f"{result['compressed_prompt'][:200]}...")
    print()
```

C.2 Measuring Compression Quality vs Token Reduction

```
from llmlingua import PromptCompressor
from openai import OpenAI
import time

compressor = PromptCompressor(
    model_name="microsoft/llmlingua-2-bert-base-multilingual-cased-meetingbank",
    use_llmlingua2=True, device_map="cuda"
)
client = OpenAI(base_url="http://localhost:8000/v1", api_key="vllm")

CONTEXT  = "..." * 500  # your RAG document
QUESTION = "Summarise the key points about distributed inference."
INSTRUCTION = "Answer based on the context only. Be concise."
GOLD_ANSWER = "..."     # expected answer for quality scoring

def run_qa(prompt_text: str, label: str) -> dict:
    t0   = time.perf_counter()
    resp = client.chat.completions.create(
        model="meta-llama/Meta-Llama-3-8B-Instruct",
        messages=[{"role": "user", "content": prompt_text}],
        max_tokens=150, temperature=0.0,
    )
    elapsed = time.perf_counter() - t0
    return {
        "label"  : label,
        "answer" : resp.choices[0].message.content,
```

```
        "tokens" : resp.usage.total_tokens,
        "cost"   : resp.usage.total_tokens / 1000 * 0.002,
        "latency": elapsed,
    }

full_prompt = f"{INSTRUCTION}

Context:
{CONTEXT}

Question: {QUESTION}"
results = [run_qa(full_prompt, "Uncompressed (1.00x)")]

for ratio in [0.5, 0.33]:
    comp = compressor.compress_prompt(
        [CONTEXT], instruction=INSTRUCTION, question=QUESTION, rate=ratio)
    comp_prompt = comp["compressed_prompt"]
    results.append(run_qa(comp_prompt, f"LLMLingua-2 ({ratio:.2f}x)"))

print(f"{'Config':<25} {'Tokens':>8} {'Cost $':>8} {'Latency':>9}")
print("-" * 55)
for r in results:
    print(f"{r['label']:<25} {r['tokens']:>8} {r['cost']:>8.4f} {r['latency']:>8.2f}s")

baseline_cost = results[0]["cost"]
for r in results[1:]:
    saving = (1 - r["cost"] / baseline_cost) * 100
    print(f"  {r['label']}: {saving:.1f}% cost reduction")
```

PART D — API Gateway, Fallback & Multi-Model Routing

Why an API Gateway?

Running vLLM directly exposed to traffic is a production anti-pattern. An API gateway sits between clients and inference backends and handles concerns that do not belong in the model server: authentication, rate limiting, request logging, SSL termination, load balancing across replicas, and routing policies. The two most common choices are **Kong** (feature-rich, plugin ecosystem) and **Traefik** (Kubernetes-native, automatic service discovery).

Concern	Without Gateway	With Gateway
Auth	vLLM has no auth -- anyone can call it	JWT/API key validated at gateway
Rate limiting	Unbounded -- one user OOMs server	Per-user/key token budget enforced
Request logging	Must instrument each model server	Single log stream at gateway
Multi-replica LB	Hard-coded URL per replica	Round-robin / least-conn at gateway
Fallback	Manual retry logic in client	Automatic failover in gateway
SSL/TLS	Each server needs cert	Terminated once at gateway
Cost attribution	No per-user token tracking	Plugin tracks tokens per API key

D.1 Traefik as LLM API Gateway (Docker Compose)

```
# docker-compose.yml -- Traefik + two vLLM replicas
version: "3.8"
services:

  traefik:
    image: traefik:v3.0
    command:
      - "--api.insecure=true"
      - "--providers.docker=true"
      - "--providers.docker.exposedbydefault=false"
      - "--entrypoints.web.address=:80"
      - "--entrypoints.metrics.address=:8082"
      - "--metrics.prometheus=true"
      - "--metrics.prometheus.entryPoint=metrics"
      - "--log.level=INFO"
      - "--accesslog=true"
    ports:
      - "80:80"
      - "8080:8080"    # Traefik dashboard
      - "8082:8082"    # Prometheus metrics
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro

  vllm-primary:
    image: vllm/vllm-openai:latest
    command: ["python", "-m", "vllm.entrypoints.openai.api_server",
```

```
              "--model", "meta-llama/Meta-Llama-3-8B-Instruct",
              "--dtype", "float16", "--port", "8000"]
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]
              device_ids: ["0"]
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.vllm.rule=PathPrefix(`/v1`)"
      - "traefik.http.routers.vllm.entrypoints=web"
      - "traefik.http.services.vllm.loadbalancer.server.port=8000"
      - "traefik.http.services.vllm.loadbalancer.healthcheck.path=/health"
      - "traefik.http.services.vllm.loadbalancer.healthcheck.interval=10s"

  vllm-secondary:
    image: vllm/vllm-openai:latest
    command: ["python", "-m", "vllm.entrypoints.openai.api_server",
              "--model", "meta-llama/Meta-Llama-3-8B-Instruct",
              "--dtype", "float16", "--port", "8000"]
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]
              device_ids: ["1"]
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.vllm.rule=PathPrefix(`/v1`)"
      - "traefik.http.services.vllm.loadbalancer.server.port=8000"

# Start: docker compose up -d
# Test:  curl http://localhost/v1/models
# Dashboard: http://localhost:8080
```

D.2 FastAPI Gateway with Auth, Rate Limiting and Logging

```
# gateway.py -- production-shaped API gateway for vLLM
from fastapi import FastAPI, HTTPException, Depends, Header
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from typing import Optional
import httpx, time, hashlib, json, asyncio
from collections import defaultdict
from datetime import datetime, timedelta

app = FastAPI(title="LLM Gateway", version="1.0")

# ■■ Config ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
BACKENDS = {
    "primary"   : "http://vllm-primary:8000",
    "secondary" : "http://vllm-secondary:8000",
    "small"     : "http://phi3-server:8000",    # cheap model for simple queries
}
VALID_API_KEYS = {"sk-prod-abc123": "user_1", "sk-prod-def456": "user_2"}
```

```
RATE_LIMIT     = 100   # requests per minute per API key
TOKEN_BUDGET   = 50_000  # tokens per hour per API key

# ■■ Rate limiter (in-memory; use Redis in production) ■■■■■■■■■■■■■■■■■■■■
request_counts: dict = defaultdict(list)
token_counts:   dict = defaultdict(int)

def check_rate_limit(api_key: str):
    now  = datetime.utcnow()
    minute_ago = now - timedelta(minutes=1)
    request_counts[api_key] = [t for t in request_counts[api_key] if t > minute_ago]
    if len(request_counts[api_key]) >= RATE_LIMIT:
        raise HTTPException(429, f"Rate limit exceeded: {RATE_LIMIT} req/min")
    request_counts[api_key].append(now)

def check_token_budget(api_key: str, tokens_used: int = 0):
    token_counts[api_key] += tokens_used
    if token_counts[api_key] > TOKEN_BUDGET:
        raise HTTPException(429, f"Token budget exceeded: {TOKEN_BUDGET}/hr")

# ■■ Auth ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
async def authenticate(authorization: Optional[str] = Header(None)) -> str:
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(401, "Missing or invalid Authorization header")
    api_key = authorization.removeprefix("Bearer ").strip()
    if api_key not in VALID_API_KEYS:
        raise HTTPException(403, "Invalid API key")
    return api_key

# ■■ Request schema ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class ChatRequest(BaseModel):
    model:      str = "auto"   # "auto" triggers complexity-based routing
    messages:   list
    max_tokens: int  = 200
    temperature: float = 0.7
    stream:     bool = False

# ■■ Simple complexity classifier ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def is_simple_query(messages: list) -> bool:
    last_user = next(
        (m["content"] for m in reversed(messages) if m["role"] == "user"), ""
    )
    # Heuristic: short questions without code/math are routed to small model
    word_count = len(last_user.split())
    has_code   = any(kw in last_user.lower()
                     for kw in ["```", "def ", "class ", "import ", "select "])
    return word_count < 30 and not has_code

# ■■ Proxy request with fallback ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
async def proxy_request(backend_url: str, payload: dict,
                        stream: bool = False) -> httpx.Response:
    timeout = httpx.Timeout(120.0)
    async with httpx.AsyncClient(timeout=timeout) as client:
        resp = await client.post(
            f"{backend_url}/v1/chat/completions",
            json=payload,
```

```
            headers={"Content-Type": "application/json"},
        )
        resp.raise_for_status()
        return resp

# ■■ Main endpoint ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@app.post("/v1/chat/completions")
async def chat_completions(req: ChatRequest, api_key: str = Depends(authenticate)):
    check_rate_limit(api_key)

    # Route to appropriate backend
    if req.model == "auto":
        backend_key = "small" if is_simple_query(req.messages) else "primary"
    else:
        backend_key = "primary"

    payload = req.model_dump()
    payload["model"] = "meta-llama/Meta-Llama-3-8B-Instruct"

    t0 = time.perf_counter()
    try:
        resp = await proxy_request(BACKENDS[backend_key], payload)
    except (httpx.HTTPError, httpx.ConnectError):
        # Fallback to secondary
        try:
            resp = await proxy_request(BACKENDS["secondary"], payload)
            backend_key = "secondary (fallback)"
        except httpx.HTTPError as e:
            raise HTTPException(503, f"All backends unavailable: {e}")

    elapsed = time.perf_counter() - t0
    data    = resp.json()

    # Track token usage
    tokens_used = data.get("usage", {}).get("total_tokens", 0)
    check_token_budget(api_key, tokens_used)

    # Structured access log
    print(json.dumps({
        "ts"         : datetime.utcnow().isoformat(),
        "api_key"    : api_key[:12] + "...",
        "backend"    : backend_key,
        "tokens"     : tokens_used,
        "latency_ms" : round(elapsed * 1000),
        "model"      : req.model,
    }))

    return data

@app.get("/health")
async def health(): return {"status": "ok", "backends": list(BACKENDS)}
```

D.3 Fallback Routing -- Chaos Test

```
# Test failover: kill primary, verify gateway falls back to secondary

import subprocess, time, requests
```

```
BASE = "http://localhost:80"

def test_endpoint(label):
    t0   = time.perf_counter()
    resp = requests.post(f"{BASE}/v1/chat/completions",
        headers={"Authorization": "Bearer sk-prod-abc123"},
        json={"messages": [{"role": "user", "content": "Say OK"}],
              "max_tokens": 5, "model": "auto"})
    elapsed = time.perf_counter() - t0
    status  = resp.status_code
    print(f"  [{label}] status={status}  elapsed={elapsed:.2f}s")
    return status == 200

# Normal operation
print("Phase 1: Normal operation")
assert test_endpoint("primary alive")

# Kill primary container (chaos)
print("
Phase 2: Killing primary backend...")
subprocess.run(["docker", "stop", "vllm-primary"], check=True)
time.sleep(2)   # wait for health check to detect failure

# Verify fallback works
print("Phase 3: Requests should fall back to secondary")
for i in range(3):
    test_endpoint(f"fallback {i+1}")

# Restore primary
print("
Phase 4: Restoring primary...")
subprocess.run(["docker", "start", "vllm-primary"], check=True)
time.sleep(15)   # wait for model to load and health check to pass

print("Phase 5: Primary restored")
assert test_endpoint("primary restored")
print("Chaos test passed.")
```

D.4 Multi-Model Routing by Query Complexity

```
# Route queries to cheap small model vs expensive large model
# based on a lightweight classifier (or simple heuristic)

import re
from openai import OpenAI
from enum import Enum

class Tier(Enum):
    SMALL = "phi3:mini"     # 3.8B -- ~$0.0002/1k tokens
    LARGE = "llama3:70b"    # 70B  -- ~$0.002/1k tokens  (10x more expensive)

def classify_query(messages: list) -> Tier:
    last_user = next(
        (m["content"] for m in reversed(messages) if m["role"] == "user"), ""
    )
    text = last_user.lower()
```

```
    # Hard signals for LARGE model
    large_signals = [
        len(last_user.split()) > 60,              # long question
        bool(re.search(r"```|def |class |import |select |join ", text)),  # code
        any(kw in text for kw in ["analyse", "compare", "explain in detail",
                                  "write a", "design", "architecture",
                                  "step by step", "pros and cons"]),
        "?" in text and len(text) > 200,           # complex question
    ]

    small_signals = [
        len(last_user.split()) <= 15,              # very short
        any(kw in text for kw in ["what is", "define", "yes or no",
                                  "translate", "spell", "how many"]),
        bool(re.match(r"^\w[\w\s]{0,30}\?$", last_user.strip())),  # simple Q
    ]

    large_score = sum(large_signals)
    small_score = sum(small_signals)

    return Tier.SMALL if small_score > large_score else Tier.LARGE

# Route and execute
def routed_completion(messages: list, max_tokens=200) -> dict:
    tier   = classify_query(messages)
    client = OpenAI(base_url="http://localhost:8000/v1", api_key="vllm")

    resp = client.chat.completions.create(
        model=tier.value, messages=messages,
        max_tokens=max_tokens, temperature=0.7,
    )
    return {
        "answer"   : resp.choices[0].message.content,
        "model"    : tier.value,
        "tier"     : tier.name,
        "tokens"   : resp.usage.total_tokens,
    }

# Test routing decisions
test_queries = [
    "What is KV caching?",
    "Analyse the trade-offs between tensor and pipeline parallelism for a 70B model "
    "that must serve 500 concurrent users with p95 TTFT under 2 seconds.",
    "Define TTFT",
    "Write a Python function that benchmarks vLLM throughput using async httpx, "
    "with configurable concurrency, reporting p50/p95/p99 latency.",
    "How many GPUs does vLLM need?",
]
for q in test_queries:
    result = routed_completion([{"role": "user", "content": q}])
    print(f"[{result['tier']:<6}] [{result['tokens']:>5} tok]  {q[:65]}")
```

Testing & Metrics

Cost Reduction Measurement Script

```
# Measure combined cost savings from all layers of the caching stack
import time, json
from collections import defaultdict

class CostTracker:
    def __init__(self, cost_per_1k=0.002):
        self.cost_per_1k = cost_per_1k
        self.events      = []

    def record(self, source: str, tokens: int, latency_ms: float):
        cost = tokens / 1000 * self.cost_per_1k
        self.events.append({"source": source, "tokens": tokens,
                            "cost": cost, "latency_ms": latency_ms})

    def report(self):
        by_source = defaultdict(lambda: {"requests": 0, "tokens": 0, "cost": 0.0})
        for e in self.events:
            s = by_source[e["source"]]
            s["requests"] += 1; s["tokens"] += e["tokens"]; s["cost"] += e["cost"]

        total_cost = sum(s["cost"] for s in by_source.values())
        # What cost would be if all went to model (no caching)
        total_toks = sum(s["tokens"] for s in by_source.values())
        # Estimate: assume all cache hits had ~200 tokens
        cached_reqs = sum(s["requests"] for k, s in by_source.items() if "cache" in k)
        hypothetical = (total_toks + cached_reqs * 200) / 1000 * self.cost_per_1k
        savings_pct  = (1 - total_cost / hypothetical) * 100 if hypothetical else 0

        print(f"{'Source':<25} {'Requests':>10} {'Tokens':>10} {'Cost':>10}")
        print("-" * 58)
        for src, s in sorted(by_source.items()):
            print(f"{src:<25} {s['requests']:>10} {s['tokens']:>10} "
                  f"${s['cost']:>9.4f}")
        print(f"
Total cost     : ${total_cost:.4f}")
        print(f"Without caching: ${hypothetical:.4f}")
        print(f"Savings        : {savings_pct:.1f}%")

tracker = CostTracker()
# ... populate tracker during request processing ...
tracker.report()
```

Key Metrics and Targets

Metric	How to Measure	Production Target
Semantic cache hit rate	cache.hits / (hits + misses)	>30% on repetitive workloads
Exact cache hit rate	redis hits / total requests	>10% for FAQ-heavy products
TTFT on cache hit	time.perf_counter() before/after	< 5ms (Redis), < 50ms (FAISS)

Prompt compression ratio	comp_tokens / orig_tokens	0.3-0.5x with LLMLingua-2
Quality delta post-compress	ROUGE-L or LLM-as-judge vs uncompressed	<5% degradation at 0.5x ratio
Routing accuracy	% queries routed to correct tier	> 90% (validate vs human labels)
Fallback success rate	fallback_ok / total_fallback_triggers	100% (zero failed fallbacks)
Gateway overhead	gateway_latency - direct_latency	< 5ms added overhead
Total cost reduction	CostTracker.report() savings %	>40% vs baseline (no caching)

Shell Verification Commands

```
# 1. Check Redis cache hit rate
redis-cli info stats | grep -E "keyspace_hits|keyspace_misses"
# hit_rate = keyspace_hits / (keyspace_hits + keyspace_misses)

# 2. Test gateway auth enforcement
curl http://localhost:80/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{"messages":[{"role":"user","content":"hi"}],"max_tokens":5}'
# Expected: 401 Unauthorized

curl http://localhost:80/v1/chat/completions \
    -H "Authorization: Bearer sk-prod-abc123" \
    -H "Content-Type: application/json" \
    -d '{"messages":[{"role":"user","content":"hi"}],"max_tokens":5}'
# Expected: 200 OK

# 3. Verify rate limiting (fire > 100 req/min)
for i in $(seq 110); do
    curl -s -o /dev/null -w "%{http_code}
" \
        http://localhost:80/v1/chat/completions \
        -H "Authorization: Bearer sk-prod-abc123" \
        -H "Content-Type: application/json" \
        -d '{"messages":[{"role":"user","content":"hi"}],"max_tokens":1}'
done | sort | uniq -c
# Expected: 100x 200, 10x 429

# 4. Traefik dashboard health
curl http://localhost:8080/api/overview | python -m json.tool

# 5. Validate semantic cache threshold
python -c "
from sentence_transformers import SentenceTransformer
import numpy as np
e = SentenceTransformer('all-MiniLM-L6-v2')
a, b = 'What is KV caching?', 'Explain KV cache mechanism'
v = e.encode([a, b], normalize_embeddings=True)
print(f'Similarity: {np.dot(v[0],v[1]):.4f}')
"
```

Key Takeaways

• **Layer your caches**: exact Redis cache (sub-millisecond, zero cost) first, then semantic FAISS cache (~5-50ms, near-zero cost), then the model (expensive). Each layer should reduce traffic to the next by 10-40%.
• **KV-level prefix caching (vLLM APC)** is free -- just enable the flag. Design system prompts to end on 16-token boundaries for maximum block alignment.
• **Semantic cache threshold matters critically**: 0.92 is a good default for technical/factual workloads. Always measure precision-recall on your specific query distribution before going to production -- 0.85 will cause quality regressions.
• **LLMLingua-2 at 0.5x ratio** halves token cost with under 5% quality loss for RAG and summarisation. Run the quality delta check on your task before deploying.
• **The API gateway enforces the contract**: auth, rate limits, token budgets, and request logging must never live in the model server. Keep vLLM pure inference.
• **Multi-model routing** captures 60-80% of cost savings in most products: simple factual queries (~50% of volume) can be answered by a 3.8B model at 1/10th the cost of a 70B. Build a routing classifier early -- even a simple heuristic beats sending everything to the large model.
• **Always chaos-test fallback routing**: kill the primary backend under load and verify the gateway's failover is automatic, fast, and transparent to clients.

Next topic: **Day 13 -- Load Testing & Full Observability**: Locust load testing at 100 concurrent users, throughput benchmarking with vLLM benchmark_serving.py, OpenTelemetry tracing, Loki log aggregation, token-level cost tracking, and A/B testing LLM responses via traffic splitting.